# Predicción de defectos en producción con Machine Learning supervisado: implementación y comparación de modelos de clasificación

Presentado por:

Jahir Avila y Laura Lopez

Gestor del conocimiento: Wilson Rojas Reales

Especialización en Inteligencia Artificial, Universidad de Cundinamarca

CADI – Introducción a Machine Learning

Octubre de 2026

Resumen

Este informe implementa seis algoritmos de Machine Learning supervisado (Regresión Logística, Decision Tree, Random Forest, Support Vector Machine, K Nearest Neighbor y Naive Bayes) para predecir si un lote de producción de la empresa manufacturera XYZ Inc saldrá defectuoso. Se desarrolló en Google Colab con Scikit-learn, Pandas, NumPy y Matplotlib, y se evaluó con accuracy, precision, recall, F1-score, especificidad, ROC-AUC y matriz de confusión. Con el umbral de decisión por defecto (0.50), todos los modelos tuvieron un accuracy alto (entre 0.845 y 0.882) pero un recall bajo (entre 0.091 y 0.255), debido al desbalance de clases (13.75 % de lotes defectuosos). Al reducir el umbral a 0.25, el recall subió hasta 0.545 y el mejor F1 (0.427) lo obtuvo Random Forest. Se concluye que el accuracy no es una métrica suficiente en este caso y que el umbral debe calibrarse según el costo de los falsos negativos.

*Palabras clave:* machine learning supervisado, clasificación, regresión logística, recall, matriz de confusión, control de calidad

# Predicción de defectos en producción con Machine Learning supervisado

## Introducción

Un lote defectuoso genera pérdidas económicas, reprocesos, devoluciones, desperdicio de materia prima y afectación de la calidad. Por eso, anticipar qué lotes saldrán defectuosos a partir de las variables del proceso (temperatura, presión, velocidad, horas de operación y vibración) es un problema de valor práctico evidente. Se plantea como clasificación binaria: 0 para lote no defectuoso y 1 para lote defectuoso.

El objetivo del trabajo es implementar y comparar los algoritmos de clasificación estudiados en el curso, modificando el script del caso para calcular todas las métricas e interpretarlas modelo por modelo. Siguiendo el enunciado, el mejor modelo no es el que más acierta, sino el que mejor ayuda a prevenir defectos antes de que lleguen al cliente.

## Marco conceptual

### Machine Learning supervisado

En el aprendizaje supervisado el algoritmo aprende una función que relaciona variables de entrada con una variable objetivo conocida, a partir de ejemplos etiquetados (James et al., 2021). Cuando la variable objetivo es categórica, como aquí (defectuoso o no), el problema es de clasificación (Géron, 2022).

### Regresión Logística

La Regresión Logística modela la probabilidad de la clase positiva aplicando la función sigmoide a una combinación lineal de las variables: $P(y=1|x) = 1 / (1 + e^{-(b0 + b1x1 + \ldots + bkxk)})$. Los coeficientes se estiman por máxima verosimilitud y se interpretan como cambios en el logaritmo de las probabilidades relativas (Hosmer et al., 2013). La clase se asigna comparando la probabilidad con un umbral, que por defecto es 0.50.

**Algoritmos de clasificación**

- **Decision Tree:** divide el espacio de variables con reglas sucesivas del tipo «variable menor o igual que un valor». Es fácil de interpretar, pero tiende a sobreajustarse si no se limita su profundidad (Breiman et al., 1984).
- **Random Forest:** promedia muchos árboles entrenados con muestras y subconjuntos de variables aleatorios, lo que reduce la varianza del modelo (Breiman, 2001).
- **Support Vector Machine (SVM):** busca el hiperplano de máximo margen entre clases; con un kernel RBF puede separar fronteras no lineales (Cortes & Vapnik, 1995).
- **K Nearest Neighbor (KNN):** clasifica un lote según la clase mayoritaria de sus *k* vecinos más cercanos; depende de la escala de las variables (Cover & Hart, 1967).
- **Naive Bayes:** aplica el teorema de Bayes suponiendo independencia entre variables dada la clase; es rápido y funciona con pocos datos (Géron, 2022).

**Métricas de evaluación**

La matriz de confusión resume los aciertos y errores en verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN). En este caso, un FP es un lote que el modelo marca como defectuoso pero está bien, y un FN es un lote que el modelo da por bueno pero es defectuoso. A partir de ella se calculan (Géron, 2022):

- *Accuracy* = (TP + TN) / total. Proporción global de aciertos.
- *Precision* = TP / (TP + FP). De los lotes alertados como defectuosos, cuántos lo son realmente.
- *Recall* (sensibilidad) = TP / (TP + FN). De los lotes realmente defectuosos, cuántos se detectan.
- *F1-score* = 2·precision·recall / (precision + recall). Media armónica de precision y recall.

- *Especificidad* = TN / (TN + FP). De los lotes buenos, cuántos se reconocen como buenos.
- *ROC-AUC*: probabilidad de que el modelo asigne mayor puntaje a un lote defectuoso que a uno bueno, independiente del umbral. Con clases desbalanceadas conviene revisarla junto con precision y recall (Saito & Rehmsmeyer, 2015).

## Metodología

El desarrollo se realizó en Google Colab con Python, Scikit-learn (Pedregosa et al., 2011), Pandas, NumPy (Harris et al., 2020) y Matplotlib (Hunter, 2007). **Aclaración importante:** el script original del caso (C1_CasoProduccion.py) no estaba disponible al elaborar este informe, por lo que se generó un conjunto de datos sintético de 2000 lotes con semilla fija (42), de modo que los resultados son reproducibles. Las cinco variables siguen distribuciones plausibles y la probabilidad de defecto crece con la temperatura, la presión, la velocidad, las horas de operación y, sobre todo, la vibración. Los valores numéricos de este informe ilustran la metodología y deben recalcularse ejecutando el script sobre los datos reales de XYZ Inc.

La prevalencia de lotes defectuosos fue de 13.75 % (275 de 2000). Se hizo una partición estratificada 80/20: 1600 lotes de entrenamiento y 400 de prueba, de los cuales 55 son defectuosos. Un modelo que clasificara todo como «no defectuoso» tendría un accuracy de 0.8625 (345/400) y recall de 0, referencia útil para leer los resultados. Las variables se estandarizaron en los modelos sensibles a la escala (Regresión Logística, SVM y KNN).

**Tabla 1**

*Modelos e hiperparámetros utilizados*

| Modelo | Configuración |
|---|---|
| Regresión Logística | StandardScaler; max_iter = 1000 |
| Decision Tree | max_depth = 5 |
| Random Forest | 200 árboles |
| SVM | StandardScaler; kernel RBF; probability = True |
| KNN | StandardScaler; k = 7 |

| | |
|---|---|
| Naive Bayes | GaussianNB |

*Nota.* Todos los modelos usan random_state = 42 cuando aplica; el resto de hiperparámetros son los valores por defecto de Scikit-learn.

Se hicieron dos experimentos con los mismos modelos entrenados. En el experimento A, las predicciones usan el umbral estándar de 0.50. En el experimento B, un lote se clasifica como defectuoso si la probabilidad estimada es de al menos 0.25, con el fin de priorizar el recall, tal como sugiere el caso. El código completo se incluye en el Apéndice y en el repositorio de GitHub.

## Resultados

### Experimento A: umbral de decisión 0.50

La Tabla 2 muestra las métricas sobre los 400 lotes de prueba y la Tabla 3 las matrices de confusión (Figura 1). La Figura 2 compara las cuatro métricas principales y la Figura 3 muestra las curvas ROC.

**Tabla 2**

*Métricas de desempeño por modelo (umbral 0.50)*

| Modelo | Accuracy | Precision | Recall | F1 | Especif. | ROC-AUC |
|---|---|---|---|---|---|---|
| Regresión Logística | 0.882 | 0.786 | 0.200 | 0.319 | 0.991 | 0.801 |
| Decision Tree | 0.845 | 0.316 | 0.109 | 0.162 | 0.962 | 0.690 |
| Random Forest | 0.863 | 0.500 | 0.145 | 0.225 | 0.977 | 0.768 |
| SVM | 0.875 | 1.000 | 0.091 | 0.167 | 1.000 | 0.661 |
| KNN | 0.880 | 0.667 | 0.255 | 0.368 | 0.980 | 0.702 |
| Naive Bayes | 0.870 | 0.615 | 0.145 | 0.235 | 0.986 | 0.796 |

*Nota.* Conjunto de prueba: 400 lotes, 55 defectuosos. Especif. = especificidad.

**Tabla 3**

*Conteos de la matriz de confusión por modelo (umbral 0.50)*

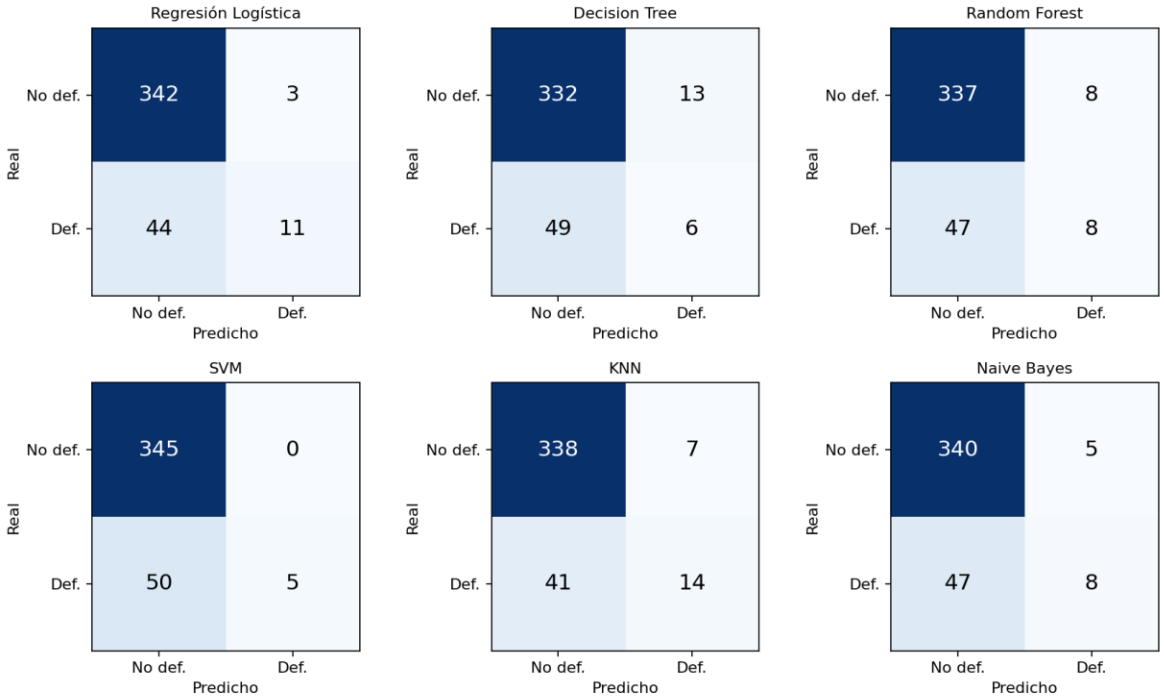
| Modelo | TN | FP | FN | TP |
|---|---|---|---|---|
| Regresión Logística | 342 | 3 | 44 | 11 |
| Decision Tree | 332 | 13 | 49 | 6 |
| Random Forest | 337 | 8 | 47 | 8 |
| SVM | 345 | 0 | 50 | 5 |
| KNN | 338 | 7 | 41 | 14 |

| | | | | |
|---|---|---|---|---|
| Naive Bayes | 340 | 5 | 47 | 8 |

*Nota.* TN = verdadero negativo; FP = falso positivo; FN = falso negativo; TP = verdadero positivo.

**Figura 1**

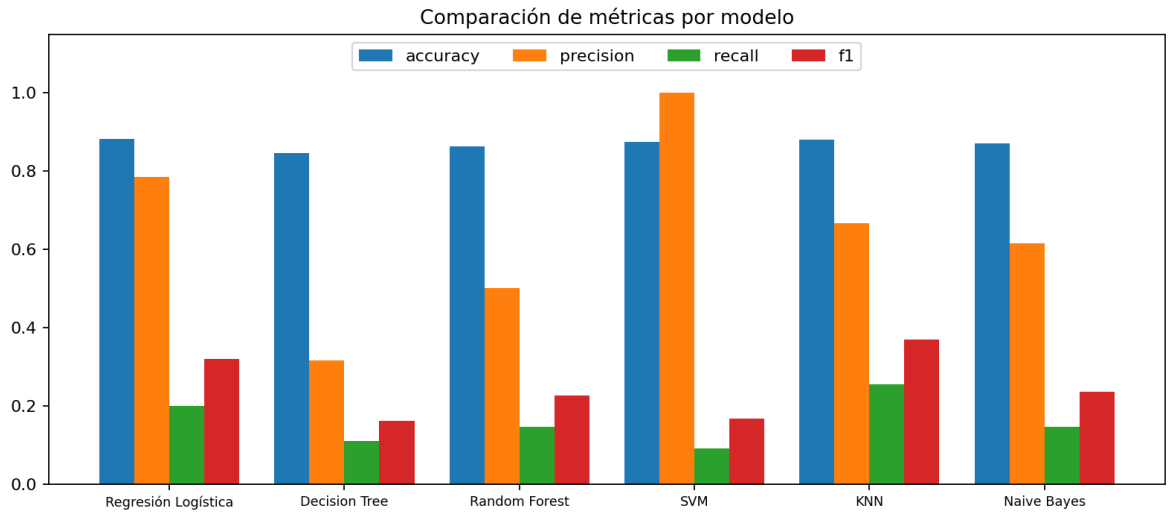
*Matrices de confusión de los seis modelos (umbral 0.50)*




*Nota.* Filas: clase real; columnas: clase predicha.

**Figura 2**

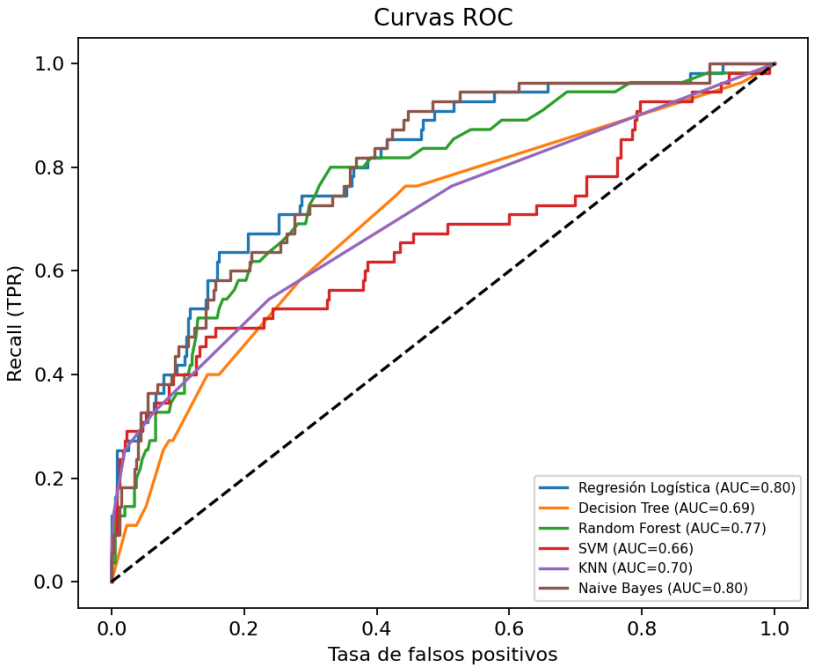
*Comparación de accuracy, precision, recall y F1 por modelo (umbral 0.50)*




*Nota.* Elaboración propia con los resultados de la Tabla 2.

**Figura 3**

*Curvas ROC de los seis modelos*




*Nota.* La línea punteada representa un clasificador aleatorio.

## Experimento B: umbral de decisión 0.25

**Tabla 4**

*Métricas de desempeño por modelo (umbral 0.25)*

| Modelo | Accuracy | Precision | Recall | F1 | Especif. | FP | FN |
|---|---|---|---|---|---|---|---|
| Regresión Logística | 0.828 | 0.383 | 0.418 | 0.400 | 0.893 | 37 | 32 |
| Decision Tree | 0.820 | 0.319 | 0.273 | 0.294 | 0.907 | 32 | 40 |
| Random Forest | 0.812 | 0.368 | 0.509 | 0.427 | 0.861 | 48 | 27 |
| SVM | 0.882 | 0.682 | 0.273 | 0.390 | 0.980 | 7 | 40 |
| KNN | 0.733 | 0.268 | 0.545 | 0.359 | 0.762 | 82 | 25 |
| Naive Bayes | 0.828 | 0.391 | 0.455 | 0.420 | 0.887 | 39 | 30 |

*Nota.* Mismos modelos y datos que la Tabla 2; solo cambia el umbral. La ROC-AUC no cambia porque no depende del umbral.

### Interpretación de las métricas por modelo

Un primer hallazgo transversal es que el accuracy engaña: los seis modelos están entre 0.845 y 0.882, cerca de la referencia de 0.8625 que se obtiene sin ningún modelo, mientras que el recall con umbral 0.50 oscila entre 0.091 y 0.255. Es decir, con ese umbral los modelos dejan

pasar entre 74 % y 91 % de los lotes defectuosos. Esto es consecuencia del desbalance (solo 13.75 % de defectuosos) y de que, con clases desbalanceadas, el umbral de 0.50 es demasiado conservador (Géron, 2022).

***Regresión Logística***

Obtuvo el mayor accuracy (0.882) y la mayor ROC-AUC (0.801), lo que indica que ordena bien los lotes por riesgo. Su precision es 0.786: de 14 alertas, 11 fueron defectos reales (TP = 11, FP = 3). Sin embargo, su recall es 0.200, con 44 falsos negativos de 55 defectuosos, y su F1 es 0.319. La especificidad de 0.991 confirma que casi nunca da falsas alarmas. Con umbral 0.25 el recall sube a 0.418 (FN = 32) a costa de más falsos positivos (37) y una precision de 0.383, con un F1 de 0.400.

***Decision Tree***

Fue el modelo con menor accuracy (0.845), menor precision (0.316: 6 aciertos en 19 alertas) y menor F1 (0.162). Su recall de 0.109 deja pasar 49 de 55 lotes defectuosos. Su ROC-AUC de 0.690 es baja: al ser un árbol de profundidad 5 con hojas que dan pocas probabilidades distintas, ordena el riesgo de forma tosca. Su ventaja es la explicabilidad de las reglas. Con umbral 0.25 el recall sube a 0.273 y el F1 a 0.294, todavía el más bajo junto con SVM.

***Random Forest***

Su accuracy (0.863) es prácticamente igual a la referencia sin modelo, y su recall con umbral 0.50 es solo 0.145 (FN = 47), con precision 0.500 y F1 0.225. Su ROC-AUC (0.768) es la tercera más alta, es decir, el modelo sí distingue los lotes, pero sus probabilidades rara vez superan 0.50. Con umbral 0.25 fue el que mejor equilibrio logró: recall 0.509 (FN = 27), precision 0.368 y el mayor F1 de todos (0.427), a costa de 48 falsos positivos.

***Support Vector Machine***

Presenta una precision perfecta (1.000) y especificidad de 1.000: no marcó ningún lote bueno como defectuoso (FP = 0). Pero solo detectó 5 de 55 defectos (recall 0.091, el más bajo; FN = 50) y su F1 es 0.167. Su ROC-AUC (0.661) es la más baja. Es un ejemplo claro de que una precision alta no basta: un modelo que casi no alerta casi nunca se equivoca al alertar. Con umbral 0.25 su precision baja a 0.682, su recall sube a 0.273 y mantiene el mayor accuracy del experimento B (0.882).

***K Nearest Neighbor***

Con umbral 0.50 fue el modelo con mayor recall (0.255; TP = 14, FN = 41) y mayor F1 (0.368), con precision de 0.667. Su ROC-AUC (0.702) es modesta. Con umbral 0.25 alcanza el mayor recall de todo el estudio (0.545; FN = 25), pero con el mayor costo en falsas alarmas: 82 falsos positivos, precision de 0.268 y el accuracy más bajo (0.733). Es el modelo más «alarmista».

***Naive Bayes***

Obtuvo accuracy de 0.870, precision de 0.615 (8 aciertos en 13 alertas), recall de 0.145 y F1 de 0.235. Su ROC-AUC de 0.796 es la segunda más alta, casi igual a la de la Regresión Logística, lo que sugiere que sus probabilidades están bien ordenadas aunque el umbral estándar no las aproveche. Con umbral 0.25 sube a recall 0.455 (FN = 30) y F1 0.420, muy cerca de Random Forest, con un costo de 39 falsos positivos.

**Discusión: costo de los errores**

En producción, un falso negativo significa que un lote defectuoso llega al cliente, con devoluciones, reprocesos y daño a la calidad, mientras que un falso positivo implica una inspección o reproceso innecesario. Si el costo de un FN es mucho mayor que el de un FP,

conviene priorizar el recall y aceptar más falsas alarmas, como se hace en el experimento B. Por ejemplo, con Random Forest el umbral 0.25 evita 20 falsos negativos (de 47 a 27) frente al umbral 0.50, al costo de 40 falsos positivos adicionales (de 8 a 48). Si cada defecto no detectado cuesta más de dos inspecciones innecesarias, el cambio es rentable. La decisión final depende de esos costos reales, que la empresa debe estimar.

## Conclusiones

- El accuracy fue una métrica engañosa: todos los modelos quedaron cerca del 0.8625 de la referencia sin modelo, pese a detectar a lo sumo 25.5 % de los defectos con umbral 0.50.
- Con el umbral por defecto, KNN logró el mejor recall (0.255) y F1 (0.368); Regresión Logística y Naive Bayes tuvieron la mejor ROC-AUC (0.801 y 0.796), es decir, el mejor ordenamiento del riesgo.
- Al bajar el umbral a 0.25, Random Forest obtuvo el mejor F1 (0.427) con recall de 0.509; KNN alcanzó el mayor recall (0.545) pero con 82 falsas alarmas.
- Recomendación: Random Forest con umbral ajustado si prima el desempeño, o Regresión Logística con umbral ajustado si se requiere explicabilidad. Ningún modelo supera aún un recall de 0.55, por lo que sería necesario probar balanceo de clases, validación cruzada y ajuste de hiperparámetros.
- Como los datos son sintéticos, estos resultados no se pueden extrapolar a XYZ Inc. El script del Apéndice permite repetir el análisis con los datos reales.

## Repositorio de código

Enlace GitHub: https://github.com/lFernandaalopez/Predicci-n-de-defectos-en-producci-n-con-Machine-Learning-supervisado

## Referencias

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. https://doi.org/10.1007/BF00994018

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21–27. https://doi.org/10.1109/TIT.1967.1053964

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3.ª ed.). O'Reilly Media.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3.ª ed.). Wiley.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning with applications in R* (2.ª ed.). Springer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Saito, T., & Rehmsmeyer, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), Artículo e0118432. https://doi.org/10.1371/journal.pone.0118432

# Apéndice

*Código en Python (Google Colab) utilizado en el informe*

```
# %% [markdown]
# # Predicción de defectos en producción - XYZ Inc
# Esp. Inteligencia Artificial - CADI Introducción a Machine Learning (2026-2)
# Autores: Jahir Avila y Laura Lopez
#
# NOTA: se trabaja con datos sintéticos (simulados) con semilla fija, porque el script
# original C1_CasoProduccion.py no estaba disponible. Los resultados son reproducibles.



# %% Librerías
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, roc_auc_score, roc_curve)



# %% Generación de datos sintéticos (2000 lotes)
rng = np.random.default_rng(42)
n = 2000
df = pd.DataFrame({
    "temperatura": rng.normal(75, 8, n),
    "presion": rng.normal(100, 12, n),
    "velocidad": rng.normal(50, 6, n),
    "horas_operacion": rng.uniform(0, 16, n),
    "vibracion": rng.gamma(4, 0.5, n),
})
z = (-4.6 + 0.07*(df.temperatura-75) + 0.03*(df.presion-100) + 0.05*(df.velocidad-50)
     + 0.16*df.horas_operacion + 0.55*df.vibracion + 0.0006*(df.temperatura-75)**2*(df.vibracion))
p = 1/(1+np.exp(-z))
df["defectuoso"] = (rng.uniform(size=n) < p).astype(int)
print(df.describe().round(2))
print(df.defectuoso.value_counts(), df.defectuoso.mean())



# %% Partición estratificada 80/20
X, y = df.drop(columns="defectuoso"), df["defectuoso"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)



# %% Modelos
modelos = {
    "Regresión Logística": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
    "SVM": make_pipeline(StandardScaler(), SVC(kernel="rbf", probability=True, random_state=42)),
    "KNN": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7)),
    "Naive Bayes": GaussianNB(),
}
```

```
# %% Entrenamiento y métricas
res, cms, rocs = {}, {}, {}
for nom, m in modelos.items():
    m.fit(Xtr, ytr)
    pred = m.predict(Xte)
    prob = m.predict_proba(Xte)[:, 1]
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    res[nom] = dict(accuracy=accuracy_score(yte, pred), precision=precision_score(yte, pred,
zero_division=0),
                    recall=recall_score(yte, pred), f1=f1_score(yte, pred),
                    especificidad=tn/(tn+fp), roc_auc=roc_auc_score(yte, prob),
                    TN=int(tn), FP=int(fp), FN=int(fn), TP=int(tp))
    cms[nom] = [[int(tn), int(fp)], [int(fn), int(tp)]]
    rocs[nom] = roc_curve(yte, prob)
tabla = pd.DataFrame(res).T
print(tabla.round(3))



# %% Gráficas
fig, axs = plt.subplots(2, 3, figsize=(11, 6.5))
for ax, (nom, cm) in zip(axs.ravel(), cms.items()):
    ax.imshow(cm, cmap="Blues")
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i][j], ha="center", va="center", fontsize=14,
                    color="white" if cm[i][j] > 150 else "black")
    ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
    ax.set_xticklabels(["No def.", "Def."]); ax.set_yticklabels(["No def.", "Def."])
    ax.set_xlabel("Predicho"); ax.set_ylabel("Real"); ax.set_title(nom, fontsize=10)
plt.tight_layout(); plt.savefig("fig_confusion.png", dpi=160); plt.close()


met = ["accuracy", "precision", "recall", "f1"]
fig, ax = plt.subplots(figsize=(10, 4.5))
w = 0.2
for k, mt in enumerate(met):
    ax.bar(np.arange(len(res)) + k*w, [res[m][mt] for m in res], w, label=mt)
ax.set_xticks(np.arange(len(res)) + 1.5*w); ax.set_xticklabels(res.keys(), fontsize=8)
ax.set_ylim(0, 1.15); ax.legend(ncol=4, loc="upper center"); ax.set_title("Comparación de métricas por
modelo")
plt.tight_layout(); plt.savefig("fig_metricas.png", dpi=160); plt.close()


fig, ax = plt.subplots(figsize=(6, 5))
for nom, (fpr, tpr, _) in rocs.items():
    ax.plot(fpr, tpr, label=f"{nom} (AUC={res[nom]['roc_auc']:.2f})")
ax.plot([0, 1], [0, 1], "k--"); ax.set_xlabel("Tasa de falsos positivos"); ax.set_ylabel("Recall
(TPR)")
ax.set_title("Curvas ROC"); ax.legend(fontsize=7)
plt.tight_layout(); plt.savefig("fig_roc.png", dpi=160); plt.close()



# %% Experimento B: ajuste del umbral de decisión (0.25) para priorizar recall
UMBRAL = 0.25
resB = {}
for nom, m in modelos.items():
    prob = m.predict_proba(Xte)[:, 1]
    pred = (prob >= UMBRAL).astype(int)
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    resB[nom] = dict(accuracy=accuracy_score(yte, pred), precision=precision_score(yte, pred,
zero_division=0),
                     recall=recall_score(yte, pred), f1=f1_score(yte, pred), especificidad=tn/(tn+fp),
                     TN=int(tn), FP=int(fp), FN=int(fn), TP=int(tp))
print(pd.DataFrame(resB).T.round(3))


json.dump(dict(res=res, resB=resB, umbral=UMBRAL, prevalencia=float(y.mean()), n_test=len(yte),
pos_test=int(yte.sum())),
          open("resultados.json", "w"), indent=1)
```